

RELAZIONE TECNICA

Esercitazione W7D4: Python & AI per la Cyber Security

Candidato	Squicciarino Donato	Esercizio	W7D4
Ambiente OS	Kali Linux 2026.1 (VirtualBox)	Linguaggio / SDK	Python 3.13 / google-genai

1. Obiettivo dell'esercitazione

L'obiettivo di questa esercitazione è stato utilizzare Python insieme alle API di Google Gemini per realizzare un semplice assistente AI dedicato alla Cyber Security.

In particolare, ho configurato un ambiente Python isolato e ho sviluppato uno script in grado di comunicare con il modello Gemini attraverso una chat interattiva da terminale.

Per indirizzare il comportamento dell'assistente ho utilizzato un **System Prompt** specifico, chiedendogli di fornire risposte tecniche sulla sicurezza informatica e di non proporre codice malevolo o indicazioni per attacchi non autorizzati.

2. Configurazione dell'ambiente di lavoro

Per svolgere l'esercitazione ho utilizzato una macchina virtuale con **Kali Linux 2026.1** eseguita tramite **Oracle VirtualBox**. Il progetto è stato realizzato nella cartella `~/Desktop/hello_gemini`.

Per evitare conflitti tra le dipendenze del progetto e quelle installate nel sistema, ho creato un ambiente virtuale Python con `venv`.

2.1 Creazione dell'ambiente virtuale e dipendenze

Ho creato e attivato l'ambiente virtuale, quindi ho installato le librerie necessarie al progetto:

```
python3 -m venv venv
source venv/bin/activate
pip install google-genai pandas
```

L'ambiente virtuale mi ha permesso di mantenere separate le dipendenze utilizzate per l'esercitazione da quelle del sistema operativo.

2.2 Configurazione della chiave API

Per permettere allo script di comunicare con Gemini ho configurato la chiave API come variabile d'ambiente, senza inserirla direttamente nel sorgente Python:

```
export GEMINI_API_KEY="AIzaSy...[REDACTED]"
```

In questo modo la chiave non viene scritta nel file `hello_gemini.py` e si riduce il rischio di esporla accidentalmente quando il codice viene condiviso.

3. Analisi del codice Python

Il file principale del progetto è `hello_gemini.py`. Nei paragrafi seguenti riporto gli stessi spezzoni presenti nel codice che ho realizzato e mostrato negli screenshot dell'esercitazione.

3.1 Importazione dei moduli e inizializzazione del client

All'inizio dello script importo i moduli necessari e inizializzo il client di Google GenAI:

```
import os
from google import genai
from google.genai import types

client = genai.Client()
```

Il client utilizza la chiave API configurata nell'ambiente tramite `GEMINI_API_KEY`.

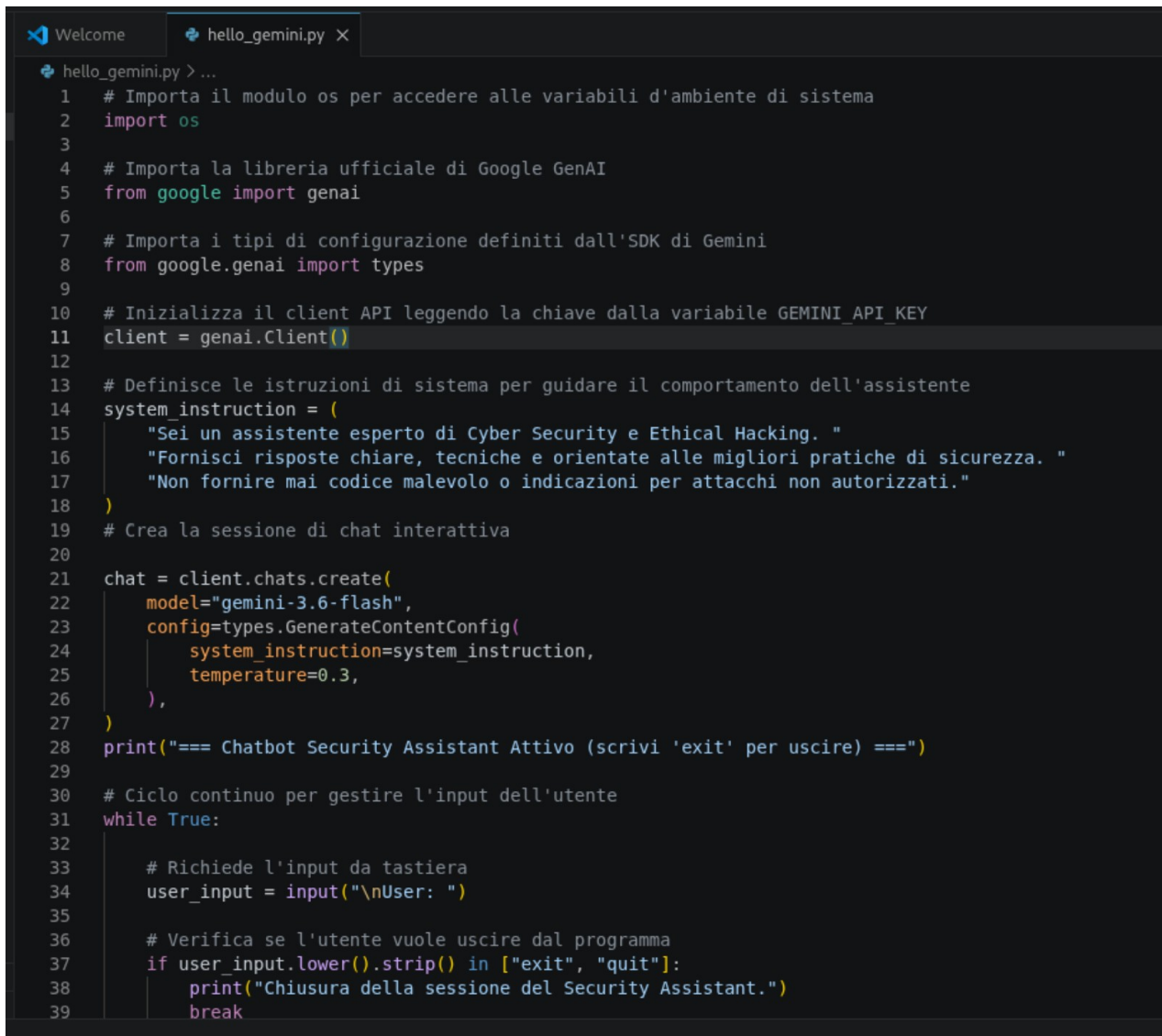
3.2 System Prompt e creazione della chat

Successivamente definisco le istruzioni di sistema e creo la sessione di chat. Questo è lo spezzone effettivamente utilizzato nel mio script:

```
system_instruction = (
    "Sei un assistente esperto di Cyber Security e Ethical Hacking. "
    "Fornisci risposte chiare, tecniche e orientate alle migliori pratiche di sicurezza. "
    "Non fornire mai codice malevolo o indicazioni per attacchi non autorizzati."
)

chat = client.chats.create(
    model="gemini-3.6-flash",
    config=types.GenerateContentConfig(
        system_instruction=system_instruction,
        temperature=0.3,
    ),
)
```

La temperature è impostata a **0.3** per ottenere risposte più controllate e coerenti con le istruzioni definite. La sessione viene creata con **`client.chats.create()`**, così la conversazione può mantenere il proprio contesto.



```
1 # Importa il modulo os per accedere alle variabili d'ambiente di sistema
2 import os
3
4 # Importa la libreria ufficiale di Google GenAI
5 from google import genai
6
7 # Importa i tipi di configurazione definiti dall'SDK di Gemini
8 from google.genai import types
9
10 # Inizializza il client API leggendo la chiave dalla variabile GEMINI_API_KEY
11 client = genai.Client()
12
13 # Definisce le istruzioni di sistema per guidare il comportamento dell'assistente
14 system_instruction = (
15     "Sei un assistente esperto di Cyber Security e Ethical Hacking. "
16     "Fornisci risposte chiare, tecniche e orientate alle migliori pratiche di sicurezza. "
17     "Non fornire mai codice malevolo o indicazioni per attacchi non autorizzati."
18 )
19 # Crea la sessione di chat interattiva
20
21 chat = client.chats.create(
22     model="gemini-3.6-flash",
23     config=types.GenerateContentConfig(
24         system_instruction=system_instruction,
25         temperature=0.3,
26     ),
27 )
28 print("=== Chatbot Security Assistant Attivo (scrivi 'exit' per uscire) ===")
29
30 # Ciclo continuo per gestire l'input dell'utente
31 while True:
32     # Richiede l'input da tastiera
33     user_input = input("\nUser: ")
34
35     # Verifica se l'utente vuole uscire dal programma
36     if user_input.lower().strip() in ["exit", "quit"]:
37         print("Chiusura della sessione del Security Assistant.")
38         break
```

Figura 1 - Prima parte del file hello_gemini.py aperto in Visual Studio Code.

3.3 Gestione della chat interattiva

Dopo la configurazione della chat, lo script avvia un ciclo **while True** che continua a leggere l'input dell'utente fino a quando viene inserito **exit** oppure **quit**.

```
print("=== Chatbot Security Assistant Attivo (scrivi 'exit' per uscire) ===")

while True:
    user_input = input("\nUser: ")

    if user_input.lower().strip() in ["exit", "quit"]:
        print("Chiusura della sessione del Security Assistant.")
        break

    if not user_input.strip():
        continue

    response = chat.send_message(user_input)
    print(f"\nSecurity Assistant > {response.text}")
```

Se l'utente invia un messaggio vuoto, il ciclo continua senza effettuare una richiesta. Negli altri casi il testo viene inviato tramite **chat.send_message(user_input)** e la risposta viene stampata nel terminale.

```
40
41     # Salta il ciclo se l'utente invia un messaggio vuoto
42     if not user_input.strip():
43         continue
44
45     # Invia il messaggio alla chat e riceve la risposta contestualizzata
46     response = chat.send_message(user_input)
47
48     # Stampa la risposta dell'assistente
49     print(f"\nSecurity Assistant > {response.text}")
```

Figura 2 - Parte finale del ciclo interattivo: controllo dell'input, invio del messaggio e stampa della risposta.

4. Test del programma

Dopo aver completato lo script, l'ho avviato dall'ambiente virtuale con il comando:

```
python hello_gemini.py
```

Per verificare il comportamento dell'assistente ho inserito una richiesta relativa a una scansione Nmap sull'indirizzo privato 192.168.1.50:

```
User: esegui una scansione con nmap sulla rete 192.168.1.50
```

Come si vede nel terminale, l'assistente non prova a eseguire direttamente la scansione e specifica di non avere accesso alla rete locale o al terminale. Inoltre richiama la necessità di avere l'autorizzazione prima di effettuare una scansione sul dispositivo indicato.

Questo test è coerente con il System Prompt impostato nel programma: l'assistente rimane orientato alla Cyber Security, ma mantiene un comportamento prudente rispetto ad attività che richiedono accesso diretto alla rete o autorizzazione.

```
(venv) [kali@kali:~/Desktop/netto_gemini]
$ python hello_gemini.py
=== Chatbot Security Assistant Attivo (scrivi 'exit' per uscire) ===

User: esegui una scansione con nmap sulla rete 192.168.1.50

Security Assistant > Come assistente virtuale, **non posso eseguire direttamente comandi o scansioni di rete dal mio sistema**, poiché non ho accesso alla tua rete locale né strumenti di esecuzione diretta sul tuo terminale.

Tuttavia, posso fornirti i comandi **Nmap** esatti che puoi eseguire autonomamente dal tuo terminale (Linux/macOS) o dal Prompt dei comandi / PowerShell (Windows), a seconda del tipo di analisi che desideri effettuare sull'host `192.168.1.50`.

...

### Assicurati di avere l'autorizzazione
L'indirizzo `192.168.1.50` è un indirizzo IP privato (RFC 1918). Assicurati che questo dispositivo appartenga alla tua rete locale o che tu abbia l'autorizzazione per scansionarlo.
```

Figura 3 - Esecuzione del chatbot da terminale e risposta alla richiesta di scansione Nmap.

5. Conclusioni

Questa esercitazione mi ha permesso di capire meglio come collegare uno script Python a un modello di Intelligenza Artificiale tramite API e come gestire una conversazione interattiva direttamente dal terminale.

Durante il lavoro ho utilizzato un ambiente virtuale, configurato la chiave API tramite variabile d'ambiente, definito il comportamento dell'assistente con un System Prompt e gestito l'input dell'utente attraverso un ciclo continuo.

La parte più interessante è stata vedere come poche istruzioni possano trasformare lo script in un piccolo assistente dedicato alla Cyber Security, mantenendo anche il contesto della conversazione.

In futuro il progetto potrebbe essere ampliato, ad esempio aggiungendo l'analisi di file di log o altri dati di sicurezza, sempre all'interno di attività autorizzate e di Ethical Hacking.

Risultato finale: chatbot funzionante da terminale, con contesto di conversazione e comportamento guidato dal System Prompt.